

Document number: P2624R0
Date: 2022-07-06
Reply-to: Justin Cooke <jgc@cems.de>
Target audience: CWG

Make operations on bools more portable

Abstract: This proposal is to resolve a contradiction in the specification of the type `bool` and to tighten the standard so as to guarantee that: (i) every object of type `bool`, regardless of provenance, has the value `true` or `false`; and (ii) conversion or promotion of `bool` to `int` always yields zero or one. The purpose is to ensure that expressions involving bools yield predictable and unsurprising results that accord with conventional mathematical logic, so that variables of type `bool` can be used safely and portably even in safety-critical applications.

Motivation: The C++20 standard (6.8.2¹⁰) states: *Type `bool` is a distinct type that has the same object representation, value representation, and alignment requirements as an implementation-defined unsigned integer type. The values of type `bool` are `true` and `false`.* However, if the `bool` type can have just two values, it cannot have the same *value* representation as the underlying unsigned integer type, which can take at least 256 distinct values. In view of this contradiction, no implementation can conform exactly to the standard; they approximate the standard in different ways, with observably different consequences. Clang implements `bool` as if it were a 1-bit bitfield contained in a 1-byte object; only the low-order bit determines the value, while the remaining bits are padding. Clang's implementation satisfies the standard's requirement that variables of type `bool` can take just two values (`true` and `false`) but a clang `bool` does not have the same value representation as the unsigned integer type that contains it. Gcc and msvc implement the `bool` type rather as if it were declared as `enum bool : unsigned char { false, true };`. The gcc and msvc implementations satisfy the standard's requirement that `bool` has the same value representation as an unsigned integer type; a gcc/msvc `bool` can take any of 256 distinct values of which just the first two have special names. On conversion or promotion to `int`, a clang `bool` always yields 0 or 1 (the padding bits are masked out), while a gcc or msvc `bool` yields its unsigned `char` value, whatever that happens to be. The gcc and msvc implementations differ from each other in their treatment of some operations on bools. The following code snippets, tested on gcc 12.1, clang 14.04 and msvc 19.32 illustrate how the three implementations can yield different results for some functions.

The purpose of this proposal is to ensure that common expressions involving bools yield predictable and unsurprising results that accord with conventional mathematical logic, so that variables of type `bool` can be used safely and portably even in safety-critical applications.

Examples of current behavior:

```
//clang always returns 0 or 1; msvc & gcc can return 0, 1 or -1
int test1(bool b)
{ switch(b)
  { case false: return 0;
    case true: return 1;
    default: return -1;
  }
}

//clang & msvc always return 1; gcc can return 1 or 2
```

```
int test2(bool b)
{ int n = 0;
  if (b) n++;
  if (!b) n++;
  return n;
}
```

//clang always returns 0 or 2; msvc & gcc can return any even value from 0 to 510

```
int test3(bool b) { return b + b; }
```

//clang & msvc always return 0 or 1; gcc can return any value from 0 to 255

```
int test4(bool b) { return b || b; }
```

Note: There are further places where the standard (N4910) is unclear concerning expressions involving bools:

Section 7.4¹ Usual arithmetic conversions [expr.arith.conv] states “*The purpose is to yield a common type, which is also the type of the result.*” This leaves it unclear whether conversions, such as integral promotions, need be performed when both operands are already of the same type as the result; for example, in the expression `a || b` where `a` and `b` are of type `bool`. Existing implementations interpret this rule differently and yield differing results for this expression.

Section 8.5.2¹ `if` statements [stmt.if] states: “*If the condition (8.5) yields true the first substatement is executed. If the else part of the selection statement is present and the condition yields false, the second substatement is executed.*” It is unclear whether the word “yields” is supposed to mean “is equal to”. As the above examples show, it is possible, given a value `b` of type `bool`, that both `b` and `!b` yield true in the context of an `if` condition, while neither `b` nor `!b` test equal to true.

Effect on existing code: The behavior of non-portable code that relies on implementation-specific features may change when the implementation is brought into conformance with the proposed new standard. The clang implementation is already in conformance with the proposed new standard.

Wording: (edits to N4910)

6.8.2 Fundamental types

[basic.fundamental]

¹⁰ Type `bool` is a distinct type that has the same object representation, ~~value representation~~, and alignment requirements as an implementation-defined unsigned integer type. The values of type `bool` are `true` and `false`. Each possible value of the unsigned integer type used for the object representation shall be interpreted either as the value `true` or as the value `false` for the type `bool`. It is implementation-dependent which values of the unsigned integer type are interpreted as `true` and which are interpreted as `false`, except that the value zero shall necessarily be interpreted as `false` and the value one shall necessarily be interpreted as `true`.

7.3.7 Integral promotions

[conv.prom]

⁶A prvalue of type `bool` can be converted to a prvalue of type `int`, with `false` becoming zero and `true` becoming one. **[Note: The result of a promotion from `bool` to `int` is always either zero or one. – end note]**

7.3.9 Integral conversions

[conv.integral]

²If the destination type is `bool`, see 7.3.15. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one. **[Note: The result of a conversion from `bool` to `int` is always either zero or one. – end note]**

Implementation notes: The proposed wording would allow an implementation to choose either a clang-like representation of bools (even number values are `false`, odd number values are `true`) or a gcc/msvc-like representation (zero is `false`, all other values “yield” `true`). However, the proposed wording imposes further requirements that the gcc and msvc implementations do not currently meet. Implementations may wish to provide a compiler warning for code whose semantics are changed relative to the existing standard. A vendor may also wish to provide a compiler switch or pragma to enable their implementation’s legacy semantics for bools, so that legacy code bases can be used as-is.

Alternatives:

- (i) A minimal resolution of the issue would be to have the standard state that undefined behavior results whenever the unsigned integer object holding a `bool` has a value other than 0 or 1. This option would obviate the need to amend any existing implementation but, to be fully portable, code would need to check the value of a `bool` of unknown provenance before using it, such as by promoting it to `int` and converting it back to `bool`.
- (ii) A maximal resolution would be to specify a common representation of bools for all implementations. This would maximize portability but would involve greater changes to some implementations’ semantics for bools.